GPU MINER OPERATIONS FIELD GUIDE

DEVOPS ENGINE

What it does · What to watch · When to act

A hands-on handbook for Bittensor GPU miners operating through the Infranex platform: how the DevOps Engine registers, validates, provisions and continuously monitors your miner — and the exact checks to run in the first ten minutes, the first hour, the first day and every week after that.

Infranex Bittensor Platform

MINER OPERATIONS · SUBNET MINING · CONTINUOUS OPS

SEPTEMBER 2026

GUIDE V1.0

CHAPTER 1

What the DevOps Engine Is

The DevOps Engine is the miner-facing control plane of the Infranex platform. It is the system that takes an empty GPU machine — a rig in your garage, a colo box, or a rented pod — and turns it into a registered, validated, provisioned and continuously supervised Bittensor miner. Everything it does is grounded in two sources of truth: the Bittensor chain (which tells it which subnet really runs where, via the on-chain identity registry) and the subnet's own GitHub repository (which tells it what the miner actually needs to run). It never invents data: a host that cannot be reached fails honestly, a subnet with no published requirements says so, and a GPU sample exists only because a real daemon reported it.

The engine is built from seven cooperating components. Each one owns a distinct part of the miner lifecycle, and each one leaves an audit trail you can inspect from the web UI or from the API. Understanding this division of labor is the fastest way to understand what the engine can do for you — and, just as importantly, what it deliberately refuses to do for you (like touching your private keys, or silently retrying a failing machine).

Component	What it actually does
Host Registry	Inventory of real SSH-reachable GPU machines. Credentials are stored AES-256-GCM encrypted and never returned to the browser. Mock/fake hosts are banned by design — only real SSH targets can be registered.
Environment Inspector	A 10-step validation pipeline run over SSH: connection, OS, GPU, driver, Docker, NVIDIA toolkit, CUDA ⩾ 12, a real GPU-in-Docker test, one-click fixes, and environment setup. Every step is persisted, resumable and shows its raw output.
Requirements Profiler	Resolves the subnet's GitHub repo from on-chain identity, scrapes its README, and derives the real install spec: minimum VRAM, CUDA/Python versions, package manager (pip / uv / Dockerfile), entrypoint, axon port, OS packages and cluster requirements. Cached 6 hours, invalidated when the override changes.

Requirement Installer	Builds a 9-step provisioning plan from that profile (packages, runtime, clone, dependencies, env file, wallet gate, launch gate, verify) and executes it one step at a time over SSH, persisting each result so the wizard is resumable and auditable.
Continuous Monitor	Every 90 seconds it evaluates every started miner: GPU health (temperature, utilization, process liveness, daemon silence) and subnet drift (hyperparameter changes, UID moves, economics regime shifts). Findings become deduplicated, approval-gated events.
Node Daemon	A tiny zero-dependency Python agent installed on the host. Every 60 seconds it HMAC-signs and uploads GPU telemetry (utilization, memory, temperature, process liveness) and pulls its command queue. It can only pull — the control plane never pushes into your machine, and every command was approved by a human.
Trigger & Strategy Layer	The recommendations inbox: GPU health, subnet drift, deregistration risk, immunity runway, upstream repo drift, benchmark regressions, validator-traffic probes and Miner-Mindset arbitrage/runtime suggestions — each with evidence and a one-click approved action.

Two design rules matter to you as an operator. First, **gates everywhere**: the engine will not spend your money or start a workload without an explicit approval from you, and it will never create, move or copy your wallet keys — the wallet step is a manual checkpoint where you confirm the files are in place. Second, **honest states**: when a daemon is silent, when a provider key is missing, or when a host fails to connect, the UI shows the failure and its remediation instead of hiding it behind a simulated green light. If you ever see a green check, a real command really ran.

CHAPTER 2

The Miner Lifecycle, End to End

Before you can monitor anything, the engine has to walk your machine through a fixed lifecycle. Knowing where your miner is in this lifecycle tells you exactly which checks apply and which alarms are meaningful. The journey has four stages, and each one ends in a state the engine records in its database — nothing is left implicit.

Stage 1 — Register the host

You give the engine a name, an IP/hostname, an SSH port, a user and one credential (password or private key). The secret is encrypted at rest and never leaves the server. Registration alone touches nothing on the machine — it only creates the inventory record, marked `PENDING` .

Stage 2 — Validate the environment (the 10-step pipeline)

The engine SSHes in and runs its inspection: **1** connect → **2** detect OS → **3** detect NVIDIA GPU (model + VRAM) → **4** validate driver and its CUDA capability → **5** validate Docker and Compose → **6** validate the NVIDIA container toolkit → **7** check CUDA ⩾ 12.0 → **8** prove GPU passthrough with a real `docker run --gpus all` test → **9** install or fix what is missing (safe fixes can run automatically) → **10** write the environment file. Each step keeps its raw output, duration and remediation hint; a fix that passes its re-check is marked FIXED .

Stage 3 — Stage the install plan

You pick a subnet and name your wallet and hotkey. The engine pulls that subnet's live requirements profile and builds a 9-step plan tailored to the repo — a venv pipeline for pip repos, a Docker pipeline for repos that ship a Dockerfile, and honest manual gates for Kubernetes-class subnets that must be provisioned with their own tooling. Staging runs nothing; the plan sits in STAGED until you execute steps.

Stage 4 — Execute, launch and monitor

You run the steps in order. Two of them are deliberately yours to control: the wallet checkpoint and the launch. When the final verify step passes, the install is DEPLOYED , the miner appears on the DevOps cockpit, and the 90-second monitor pass starts watching it for the rest of its life. You can stop the miner at any time (a safety stop disables the service and marks the install STOPPED), and relaunching is a one-step re-run.

The three gate types you will meet in the plan are worth memorizing, because they define who is allowed to do what:

Gate	Meaning	Which steps
AUTO	Runs unattended when you click the step; output is recorded verbatim.	OS packages, venv/Docker setup, repo clone, dependencies, env file, verify.
MANUAL	The engine can check, but only you can truthfully confirm — it refuses to pretend.	Wallet files present on the host (your coldkey + hotkey, copied by you).
APPROVAL	Starts burning compute; requires an explicit human approval click.	Launch the miner (systemd unit or GPU container).

CHAPTER 3

What to Check Once the GPU Miner Is Started

The moment the launch step completes, your job changes from building to watching. The engine watches with you — every 90 seconds, automatically — but the division of labor is specific: the engine detects and recommends, **you** confirm and approve. This chapter gives you the exact check sequence in four time windows, from the first ten minutes to the ongoing weekly rhythm. Every check names where to look (the DevOps cockpit or the host itself) and what a healthy result looks like, so a green pass is never a matter of opinion.

WINDOW 1 · FIRST 10 MINUTES

Is it actually up?

- **Service state.** The verify step ran `systemctl is-active infranex-miner-sn<netuid>` and expected **active**. Re-run it yourself from the host; a unit that flaps (crash-looping under `Restart=always`) shows as activating for a while before it shows as failed.
- **Axon port listening.** The engine checks `ss -tlnp | grep :<axon-port>` (8091 for most subnets). If the port is not listening, validators cannot reach you and no incentive can flow, no matter how healthy the process looks.
- **First log lines are clean.** The engine tails the log (systemd path: `/var/log/infranex-miner-sn<netuid>.log`; Docker path: `docker logs --tail 20 infranex-miner-sn<netuid>`). Look for the wallet being loaded, the network (finney) being connected and the axon serving — not for tracebacks, wallet-not-found or chain-sync errors.
- **Daemon telemetry arrives.** On the DevOps cockpit your miner card should flip the daemon badge to online within a minute or two (the daemon posts every 60 seconds). If it stays offline, the platform has no eyes on the machine and every later alarm is blind.
- **The monitor pass sees you.** The cockpit footer shows the last monitor pass; within one or two passes (⩽ 3 minutes) your miner should be counted in it.

```
# on the host — the same evidence the verify step collects
systemctl is-active infranex-miner-sn<netuid>
ss -tlnp | grep ':<axon-port> '
tail -n 20 /var/log/infranex-miner-sn<netuid>.log
# docker-installed repos instead:
docker ps --filter name=infranex-miner-sn<netuid>
docker logs --tail 20 infranex-miner-sn<netuid>
```

WINDOW 2 · FIRST HOUR

Is it healthy and reachable?

- **GPU vitals.** The miner card's sparkline should show real samples: temperature comfortably below the 78 °C warning line, utilization following the workload, and VRAM allocated against capacity. A flat zero utilisation on a warm GPU means the process is alive but not working.
- **Chain registration.** The engine reads your UID, incentive and emission straight from the metagraph. A fresh miner shows its assigned UID; if the UID column stays empty your hotkey is not registered on this subnet and nothing you fix on the host will change that.
- **Axon probe.** The service-health pass synthetically probes your axon from the platform side. A `PROBE FAIL` here with a healthy local service usually means the port is published but not reachable (firewall, NAT, provider security group).
- **Recommendations inbox is quiet.** One or two passes can surface early problems (thermal, daemon silence). A critical event in the first hour is normal to see once — it is not normal to ignore.

WINDOW 3 · FIRST 24 HOURS

Is it earning and stable?

- **Incentive is flowing.** After validators score you, the incentive/emission figures on the miner card should move off zero. Emissions lag registration by some hours — that is chain economics, not a platform fault.
- **No repeated thermal or idle warnings.** The engine only alarms after repeated confirmations (two passes for temperature, three for idle GPU), so any warning that survives a full day is a sustained condition, not noise.
- **Registration is stable.** Watch for `DEREG-RISK` or `RUNWAY` events: hotkey UID moves or immunity-period countdowns mean your slot on the subnet is contested.
- **No unexplained drift.** `SUBNET DRIFT` events in the first day usually mean the subnet owner changed hyperparameters or the metagraph moved — the event carries the evidence; approve the suggested re-sync or restart when it looks right.

WINDOW 4 · ONGOING (DAILY / WEEKLY)

Keep it earning

- **Daily — triage the inbox.** Every open event carries evidence and a suggested action (restart miner, re-sync config, recycle compute, fail over). Approve what is right, decline what is not; either way the event history is your ops log.
- **Daily — glance at the sparklines.** Temperature creep (dust, fan failure) and utilisation collapse show up hours before earnings stop.
- **Weekly — upstream watch.** The engine diffs the subnet's repo; `UPSTREAM` events mean the code you are running is no longer what the subnet ships. `BENCH REGRESS` events mean your measured performance fell against the rolling baseline.
- **Weekly — housekeeping.** The systemd unit appends to `/var/log/infranex-miner-sn<netuid>.log` forever — check disk space. The requirements profile re-scrapes automatically every 6 hours, so a subnet that changes its README re-plans without your involvement.
- **Weekly — provider keys.** Provisioning actions that spin up new machines need live provider API keys (RunPod / Hetzner / DigitalOcean). A missing key surfaces as an honest configuration error, not a silent failure — re-add keys in Settings when prompted.

The what-and-when matrix

When	Check	Healthy looks like	Where
First 10 min	Service active + axon port listening	**active** + port in LISTEN	Host / verify step
	First log lines	Wallet loaded, axon serving, no tracebacks	Host log tail
	Daemon telemetry + monitor pass	Daemon online; miner counted in pass	DevOps cockpit
First hour	GPU temp / util / VRAM samples	Temp < 78 °C; util follows workload	Miner card sparkline
	UID / registration on chain	UID assigned	Miner card chain facts
	Axon reachability probe	No PROBE FAIL event	Recommendations inbox
First 24 h	Incentive / emission flow	Moves off zero	Miner card economics

First 24 h	Sustained alarms	No thermal / idle / dereg events	Recommendations inbox
Daily	Open events triage	All events approved or declined	Recommendations inbox
	Sparkline trends	No temp creep, no util collapse	Miner Ops board
Weekly	Upstream repo / benchmark drift	No unresolved UPSTREAM / BENCH events	Change feed
	Log disk usage + provider keys	Disk under control; keys valid	Host + Settings

CHAPTER 4

Alarm Thresholds & Event Reference

The monitor enforces one fixed set of thresholds, published on the cockpit so the UI always draws the same lines the engine enforces. Two properties of these alarms matter for interpreting them. First, **debounce by design**: a condition must hold for several consecutive 90-second passes before it becomes an event, so single-sample spikes never page you. Second, **evidence in every event**: each finding stores the readings that triggered it, and recovery auto-resolves the event when the condition clears, so the change feed is a faithful history rather than a pile of stale tickets.

Threshold	Value	Meaning
GPU temperature warning	**78 °C** × 2 passes	Sustained hot GPU — check cooling before it throttles.
GPU temperature critical	**85 °C** × 2 passes	Danger zone — expect throttling and possible hardware damage.
Idle GPU	util < **30%** × 3 passes	Process alive but not doing subnet work — typically a stuck or mis-configured miner.
Miner process dead	**2 passes**	Critical event; the suggested action is a daemon `restart_miner` command.
Daemon silence	**10 minutes**	No telemetry for 10 min — daemon down, host down, or network path broken.

Economics regime shift	**±50%**	Median rewarded incentive on the subnet moved by half — re-check profitability.
Sample retention	**240 samples**	≈ 6 hours of GPU history at the 90-second cadence (sparkline window).

Event kinds you will meet in the inbox

Event	What it tells you
GPU HEALTH	Thermal, idle or process-down condition on a specific miner, with the readings attached.
SUBNET DRIFT	The subnet changed under you: hyperparameters (tempo, immunity, UID cap), metagraph capacity, economics regime, or your hotkey's UID moved.
DEREG-RISK	Your registration slot is at risk — typically UID churn; the action is usually failover to a fallback profile.
RUNWAY	Immunity-period countdown entered the watch or at-risk band: deregistration becomes possible at T-zero.
PROBE FAIL	The synthetic axon probe could not reach your miner from the platform — network/port problem, not process health.
SERVICE LATENCY	Your axon responds but too slowly — validators may be timing you out.
QUERY DROUGHT	Validator traffic to your miner dried up — you are registered but not being queried.
ESCALATION	The automatic repair ladder is exhausted; approving executes the recorded rollback (or the next step is a stop).
UPSTREAM	The subnet's repo moved after you deployed — your running code is stale.
BENCH REGRESS	A benchmark run regressed against the rolling baseline — performance, not uptime.
ARBITRAGE	Miner-Mindset finding: another subnet pays better for your GPU class; the action recycles compute.
RUNTIME OPT	Miner-Mindset finding: a serving profile (accelerated runtime recipe) fits your workload and would raise yield.

CHAPTER 5

Step-by-Step Operator Routine

This is the distilled daily loop, written as you would actually perform it. The first three steps happen in the web UI; the rest only apply when something looks wrong or when you prefer to confirm on the machine itself. Done faithfully, the whole routine takes a few minutes a day per miner — the engine has already done the watching; you are doing the judging.

1. **Open the DevOps cockpit and read the summary bar.** It states in one line how many miners are monitored and how many are healthy, warning or critical, whether daemons are online, and the current infra cost versus revenue per day. This bar is the fastest honest answer to "is everything fine right now?"

2. **Triage the recommendations inbox.** Work top-down: critical events first (GPU health, escalations, probe failures), then warnings (drift, deregistration risk, runway), then strategy findings (arbitrage, runtime optimizations). Expand each event, read its evidence, and either apply the suggested action to the GPU or decline it — never leave criticals sitting.

3. **Walk the per-miner ops board.** For each started miner, check the daemon badge (online, last seen within a couple of minutes), the GPU sparklines (temperature and utilization trends over the last ~6 hours), and the chain facts (UID, incentive, emission, registration state). Anything flatlined or empty gets investigated in steps 4–6.

4. **Confirm the process is really serving.** On the host, the unit and its log answer in seconds. The systemd unit is named `infranex-miner-sn<netuid>` and the Docker container reuses the same name, so one set of commands covers both install paths.

```
# process + port + recent log, one shot
systemctl status infranex-miner-sn<netuid> --no-pager -l
journalctl -u infranex-miner-sn<netuid> -n 50 --no-pager
tail -n 50 /var/log/infranex-miner-sn<netuid>.log
nvidia-smi
```

5. **If the daemon is unreachable, check the agent, not just the miner.** The daemon is a small Python service posting every 60 seconds. Silence beyond 10 minutes triggers the platform warning; the usual culprits are the daemon process itself, the host's outbound network, a platform URL change, or clock skew (signatures are only valid within a ±5-minute window, so a wrong clock silently kills telemetry). Restarting the daemon process is safe and requires no re-registration.

6 **When an action fires, verify its effect.** Approved commands (restart miner, apply config) reach the daemon's queue on its next poll and the result is recorded back to the event. After any restart or re-sync, re-check the miner card: telemetry resumes, the process-alive flag flips back, and the event auto-resolves when the underlying condition clears. If nothing changes within a few passes, the problem is on the host — go to step 4.

7 **Close the loop weekly.** Review the change feed for what auto-resolved while you were away, check log disk usage on long-running hosts, confirm provider keys in Settings are still valid for machines the engine may provision, and skim upstream/benchmark events to decide whether a re-deploy to newer subnet code is warranted.

CHAPTER 6

Troubleshooting Map

When an install step fails, the engine writes a remediation hint next to the raw output — the table below collects the ones you will actually meet, so you can act without deciphering raw logs first. The pattern to remember: install failures are almost always environment problems (network, apt sources, missing prerequisites) on the host, and they are re-runnable — fix the cause, press the step again, and the plan resumes where it stopped. Nothing ever continues past a failed step on its own.

Failed step	Typical cause	What to do
Install OS packages	Broken apt sources or no outbound network on the host.	Fix distro sources / connectivity, re-run the step.
Install Docker + NVIDIA runtime	curl or the NVIDIA repo unreachable from the host.	Install docker-ce and nvidia-container-toolkit manually, then retry.
Create Python venv	Python version mismatch or python3-venv missing.	Install the required python3.x (deadsnakes) — the OS-packages step normally adds venv.
Clone the subnet repo	Repo not public or host has no egress.	Check the URL is reachable from the host; retry.
Install Python dependencies	Heavy CUDA wheels; RAM or disk pressure.	Read the pip output, free resources if needed, retry.
Wallet files present	Coldkey/hotkey not copied to the host.	Copy `~/.bittensor/wallets/<wallet>` (coldkey + hotkeys/<hotkey>) to the host via scp, then confirm.

Launch the miner	Bad wallet path, port conflict, chain error.	The verify step's log tail usually shows the reason; fix and re-run the launch step.
Verify the miner is live	Service inactive or axon port not listening.	`journalctl -u infranex-miner-sn<netuid>` — common causes: wallet path, port conflict, chain sync.

Two platform-side conditions round out the map. **Mock transport refused**: if you ever import or inherit a host record marked as a mock, every step will fail loudly with a message telling you to delete and re-register the machine over real SSH — that is the honesty rule working as designed, not a bug. **Missing provider keys**: flows that create machines on your behalf (deployments migrating into DevOps, autopilot recycling compute) will fail with a clear configuration error until the RunPod / Hetzner / DigitalOcean keys are re-added in Settings; the engine will never fall back to a simulated machine to cover the gap.

Real machines.
Real telemetry.
Nothing fabricated.

The DevOps Engine does the watching so you can do the judging: it registers, validates, provisions and monitors every miner you start — but it never spends your compute, moves your keys, or hides a failure. When you see green, a real command really ran.

POCKET REFERENCE

unit name	infranex-miner-sn<netuid>
log file	/var/log/infranex-miner-sn<netuid>.log
monitor cadence	every 90 s
daemon heartbeat	every 60 s (silence > 10 min = warning)
thermal alarms	78 °C warn / 85 °C crit (×2 passes)
idle alarm	util < 30% × 3 passes
requirements cache	re-scraped every 6 h